**Laxmi Charitable Trust's**
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Practical

# Practical – VIII

| Roll No. : T045 | Name: Omkar Thukarul |
|---|---|
| Class : TYIT | Date/Time of Submission : 22/08/2026 |

## Aim: Ethics and Bias Analysis

**1.Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class Imbalance.**

**Dataset: Titanic train.csv from Kaggle**

**CODE:**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# 1. Load and Explore the Dataset
df = pd.read_csv("Titanic_train.csv")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nDataset Information:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
# 2. Check Data Quality
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
# 3. Identify the Target Variable
target = "Survived"
print("\nTarget Variable:", target)
print("\nTarget Values:")
print(df[target].value_counts())
# 4. Analyze Class Distribution
class_distribution = df[target].value_counts()
print("\nClass Distribution:")
print(class_distribution)
print("\nClass Distribution Percentage:")
print(df[target].value_counts(normalize=True) * 100)
# 5. Visualize Class Imbalance
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Survived")
```

***OUTPUT:***

```
First 5 rows:
   PassengerId  Survived  Pclass       Name   Sex   Age  SibSp  Parch      Ticket   Fare Cabin Embarked
0            1         1       3  Passenger_1  male   6.1      0      0  TICKET00001  14.06   NaN        C
1            2         0       3  Passenger_2  male  30.9      0      0  TICKET00002   7.55   NaN        S
2            3         1       3  Passenger_3  male  37.6      0      1  TICKET00003  17.30  E46        C
3            4         0       3  Passenger_4  male   NaN      2      0  TICKET00004  23.28  B20        Q
4            5         0       3  Passenger_5  male  26.9      0      0  TICKET00005  16.63  C85        S

Dataset Shape:
(891, 12)

Column Names:
['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked']

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        329 non-null    str
 11  Embarked     845 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
None

Statistical Summary:
       PassengerId    Survived      Pclass         Age      SibSp      Parch       Fare
count   891.000000  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean    446.000000    0.382806    2.296296   30.352661    0.542888    0.396184   37.714736
std     257.353842    0.500273    0.859695   14.006024    0.966385    0.885282   44.676618
min       1.000000    0.000000    1.000000    0.500000    0.000000    0.000000    2.660000
25%     223.500000    0.000000    1.000000   21.025000    0.000000    0.000000   11.040000
50%     446.000000    1.000000    3.000000   30.700000    0.000000    0.000000   17.990000
75%     668.500000    1.000000    3.000000   39.675000    1.000000    1.000000   48.925000
max     891.000000    1.000000    3.000000   80.000000    5.000000    4.000000  453.520000

Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          562
Embarked        46
dtype: int64

Duplicate Rows:
0
```

```
plt.title("Titanic Survival Class Distribution")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Number of Passengers")
plt.show()
# 6. Analyze Gender Representation
print("\nGender Representation:")
print(df["Sex"].value_counts())
print("\nGender Percentage:")
print(df["Sex"].value_counts(normalize=True) * 100)
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Sex")
plt.title("Gender Representation")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")
plt.show()
# 7. Analyze Survival by Gender
gender_survival = pd.crosstab(df["Sex"], df["Survived"])
print("\nSurvival by Gender:")
print(gender_survival)
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Sex", hue="Survived")
plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.show()
# Survival percentage by gender
gender_survival_rate = pd.crosstab(
    df["Sex"],
    df["Survived"],
    normalize="index"
) * 100
print("\nSurvival Percentage by Gender:")
print(gender_survival_rate)
# 8. Analyze Passenger-Class Representation
print("\nPassenger Class Representation:")
print(df["Pclass"].value_counts().sort_index())
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Pclass")
plt.title("Passenger-Class Representation")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.show()
# 9. Analyze Survival by Passenger Class
class_survival = pd.crosstab(df["Pclass"], df["Survived"])
print("\nSurvival by Passenger Class:")
print(class_survival)
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Pclass", hue="Survived")
plt.title("Survival by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.legend(title="Survived", labels=["No", "Yes"])
plt.show()
class_survival_rate = pd.crosstab(
    df["Pclass"],
```

```
Data Types:
PassengerId      int64
Survived         int64
Pclass           int64
Name               str
Sex                str
Age            float64
SibSp            int64
Parch            int64
Ticket             str
Fare           float64
Cabin              str
Embarked           str
dtype: object

Target Variable: Survived

Target Values:
Survived
1    448
0    443
Name: count, dtype: int64

Class Distribution:
Survived
1    448
0    443
Name: count, dtype: int64

Class Distribution Percentage:
Survived
1    50.280584
0    49.719416
Name: proportion, dtype: float6

Gender Representation:
Sex
male      603
female    288
Name: count, dtype: int64

Gender Percentage:
Sex
male      67.676768
female    32.323232
Name: proportion, dtype: float6

Survival by Gender:
Survived    0    1
Sex
female     59  229
male      384  219

Survival Percentage by Gender:
Survived          0          1
Sex
female    20.486111  79.513889
male      63.681592  36.318408

Passenger Class Representation:
Pclass
1    236
2    155
3    500
Name: count, dtype: int64
```

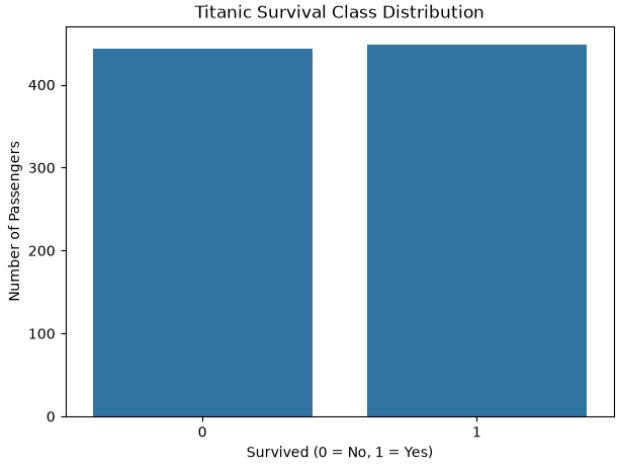
```
Survival by Passenger Class:
Survived    0    1
Pclass
1          74  162
2          82   73
3         287  213

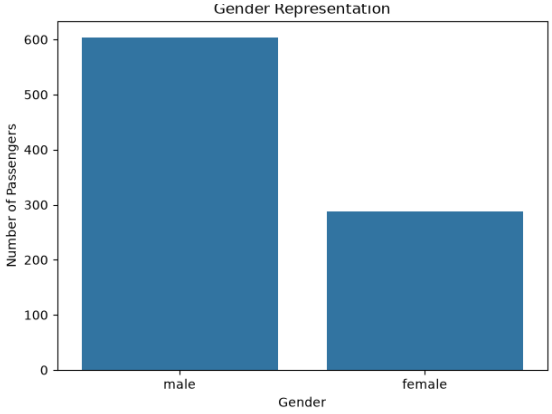
Survival Percentage by Passenger Class:
Survived          0          1
Pclass
1         31.355932  68.644068
2         52.903226  47.096774
3         57.400000  42.600000

Age Group Distribution:
AgeGroup
Child          73
Teenager       65
Young Adult    206
Adult          314
Senior         56
Name: count, dtype: int64

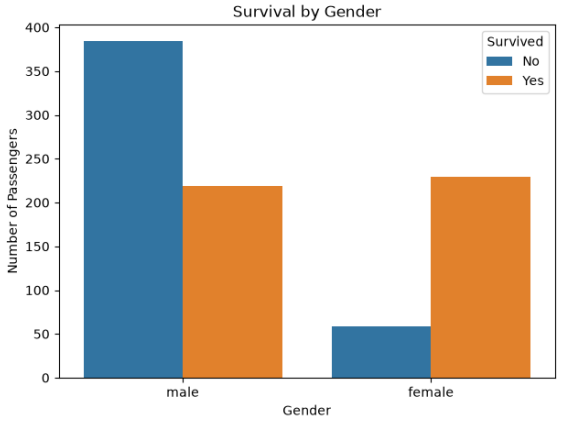
Survival by Age Group:
Survived       0    1
AgeGroup
Child         23   50
Teenager      23   42
Young Adult   97  109
Adult        177  137
Senior        34   22

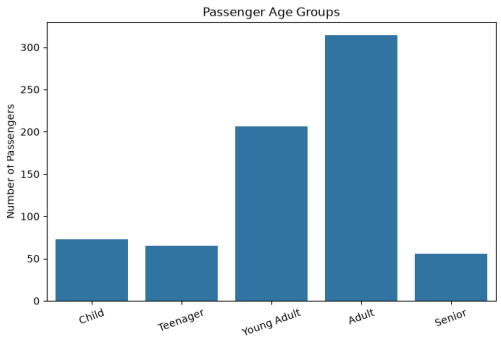
Survival Percentage by Age Group:
Survived           0          1
AgeGroup
Child          31.506849  68.493151
Teenager       35.384615  64.615385
Young Adult    47.087379  52.912621
Adult          56.369427  43.630573
Senior         60.714286  39.285714
```

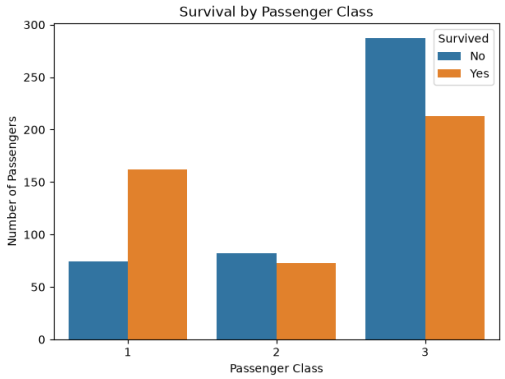


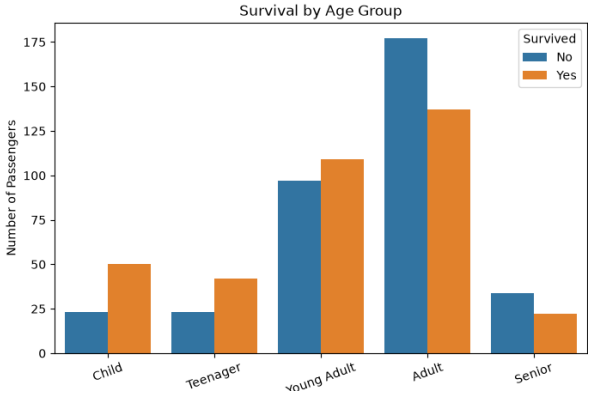



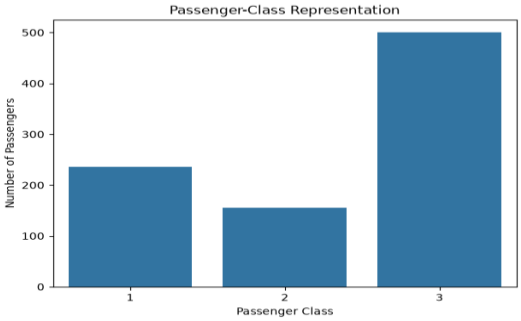
```
    df["Survived"],
    normalize="index"
) * 100
print("\nSurvival Percentage by Passenger Class:")
print(class_survival_rate)
# 10. Analyze Age Groups
# Remove missing ages for age analysis
age_df = df.dropna(subset=["Age"]).copy()
bins = [0, 12, 18, 30, 50, 100]
labels = ["Child", "Teenager", "Young Adult", "Adult", "Senior"]
age_df["AgeGroup"] = pd.cut(
    age_df["Age"],
    bins=bins,
    labels=labels,
    right=True
)
print("\nAge Group Distribution:")
print(age_df["AgeGroup"].value_counts().sort_index())
plt.figure(figsize=(8, 5))
sns.countplot(
    data=age_df,
    x="AgeGroup",
    order=labels
)
plt.title("Passenger Age Groups")
plt.xlabel("Age Group")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=20)
plt.show()
# 11. Analyze Survival by Age Group
age_survival = pd.crosstab(
    age_df["AgeGroup"],
    age_df["Survived"]
)
print("\nSurvival by Age Group:")
print(age_survival)
plt.figure(figsize=(8, 5))
sns.countplot(
    data=age_df,
    x="AgeGroup",
    hue="Survived",
    order=labels
)
plt.title("Survival by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Passengers")
plt.xticks(rotation=20)
plt.legend(title="Survived", labels=["No", "Yes"])
plt.show()
age_survival_rate = pd.crosstab(
    age_df["AgeGroup"],
    age_df["Survived"],
    normalize="index"
) * 100
print("\nSurvival Percentage by Age Group:")
print(age_survival_rate)
```






## 12. Identify Possible Bias

Based on the analysis, possible bias can be identified from differences in survival rates.

- **Gender bias:** Survival rates differ significantly between males and females.
- **Passenger-class bias:** First-class passengers generally had better survival outcomes than second- and third-class passengers.
- **Age bias:** Children and adults may have different survival probabilities.
- **Missing-age bias:** A significant number of passengers have missing `Age` values.
- **Historical/social bias:** Titanic survival was influenced by social class, gender, access to lifeboats, and location on the ship.

## 13. Impact on Machine Learning

These biases can affect an ML model in several ways:

1. The model may learn historical inequalities rather than general patterns.
2. Predictions can become strongly dependent on gender or passenger class.
3. Missing values can reduce prediction accuracy.
4. An imbalanced target variable can make accuracy misleading.
5. A model may perform well overall but poorly for a particular demographic group.

## 14. Suggested Solutions

- Handle missing `Age` values using median imputation or an appropriate model-based method.
- Check class imbalance before training.
- Compare performance separately for males and females.
- Use metrics such as **precision, recall, F1-score, and confusion matrix**, not only accuracy.
- Consider fairness metrics such as demographic parity or equal opportunity.
- Avoid unnecessarily using sensitive attributes if they are not required.
- Use stratified train-test splitting.
- Apply resampling/class-weight techniques if class imbalance becomes significant.

## 15. Final Conclusion

The Titanic dataset demonstrates that survival was strongly associated with factors such as **gender, passenger class, and age**. The target variable is not perfectly balanced, and substantial differences exist between demographic groups. These differences are useful for prediction but can also introduce bias into machine-learning models. Therefore, data quality, class distribution, subgroup performance, and fairness should be evaluated before deploying a predictive model.

## 2)Dataset: German Credit Risk Dataset from Kaggle

**CODE:**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# 1. Load and Explore the Dataset
df = pd.read_csv("german_credit_data.csv")
print("First 5 rows:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nColumn Names:")
print(df.columns.tolist())
print("\nDataset Information:")
```

***OUTPUT:***

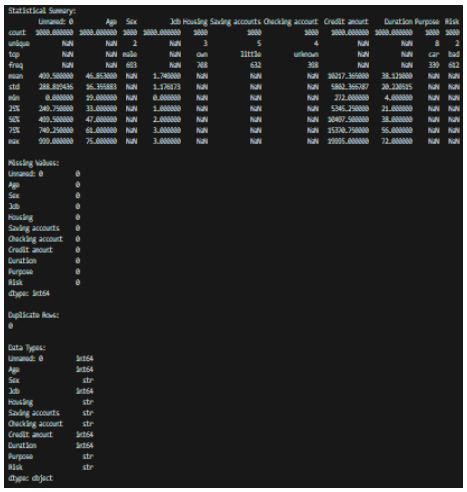
```
First 5 rows:
   Unnamed: 0  Age     Sex  Job Housing Saving accounts Checking account  Credit amount  Duration  Purpose  Risk
0           0   71    male    3     own          little          moderate           1348        55  radio/TV  good
1           1   71    male    3     own          little          unknown           12803        32  business   bad
2           2   75    male    1    rent         unknown             rich            7827        65       car  good
3           3   73    male    1     own          little          unknown           4635         5  radio/TV   bad
4           4   67  female    1    rent        moderate          little           11207        33  business   bad

Dataset Shape:
(1000, 11)

Column Names:
['Unnamed: 0', 'Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Purpose', 'Risk']

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Unnamed: 0        1000 non-null   int64
 1   Age               1000 non-null   int64
 2   Sex               1000 non-null   str
 3   Job               1000 non-null   int64
 4   Housing           1000 non-null   str
 5   Saving accounts   1000 non-null   str
 6   Checking account  1000 non-null   str
 7   Credit amount     1000 non-null   int64
 8   Duration          1000 non-null   int64
 9   Purpose           1000 non-null   str
 10  Risk              1000 non-null   str
dtypes: int64(5), str(6)
memory usage: 86.1 KB
```

```
df.info()
print("\nStatistical Summary:")
print(df.describe(include="all"))
# 2. Check Data Quality
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:")
print(df.duplicated().sum())
print("\nData Types:")
print(df.dtypes)
# 3. Identify Target Variable
target = "Risk"
print("\nTarget Variable:", target)
print("\nTarget Values:")
print(df[target].value_counts())
# 4. Analyze Class Distribution
risk_distribution = df[target].value_counts()
print("\nCredit Risk Class Distribution:")
print(risk_distribution)
print("\nCredit Risk Percentage:")
print(df[target].value_counts(normalize=True) * 100)
# 5. Visualize Class Imbalance
plt.figure(figsize=(7,5))
sns.countplot(data=df, x="Risk")
plt.title("Credit Risk Class Distribution")
plt.xlabel("Credit Risk")
plt.ylabel("Number of Customers")
plt.show()
# 6. Analyze Gender Representation
print("\nGender Representation:")
print(df["Sex"].value_counts())
print("\nGender Percentage:")
print(df["Sex"].value_counts(normalize=True) * 100)
plt.figure(figsize=(7,5))
sns.countplot(data=df, x="Sex")
plt.title("Gender Representation")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")
plt.show()
# 7. Analyze Credit Risk by Gender
gender_risk = pd.crosstab(df["Sex"], df["Risk"])
print("\nCredit Risk by Gender:")
print(gender_risk)
plt.figure(figsize=(7,5))
sns.countplot(data=df, x="Sex", hue="Risk")
plt.title("Credit Risk by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Customers")
plt.show()
gender_risk_percentage = pd.crosstab(df["Sex"], df["Risk"],
 normalize="index") * 100
print("\nCredit Risk Percentage by Gender:")
print(gender_risk_percentage)
# 8. Analyze Age Groups
bins = [18,25,35,50,65,100]
labels = ["18-25","26-35","36-50","51-65","66+"]
```



```
Statistical Summary:
            Unnamed: 0          Age     Sex          Job Housing Saving accounts Checking account  Credit amount     Duration Purpose  Risk
count      1000.000000  1000.000000    1000  1000.000000    1000            1000            1000    1000.000000  1000.000000    1000  1000
unique             NaN          NaN       2          NaN       3               5               4            NaN          NaN      8     2
top                NaN          NaN    male          NaN     own          little      moderate            NaN          NaN     car   bad
freq               NaN          NaN     690          NaN     713             603             274            NaN          NaN     337   700
mean        499.500000    35.546000     NaN     1.904000     NaN             NaN             NaN    3271.258000    20.903000     NaN   NaN
std         288.819436    11.375469     NaN     0.653614     NaN             NaN             NaN    2822.736876    12.058814     NaN   NaN
min           0.000000    19.000000     NaN     0.000000     NaN             NaN             NaN     250.000000     4.000000     NaN   NaN
25%         249.750000    27.000000     NaN     2.000000     NaN             NaN             NaN    1365.500000    12.000000     NaN   NaN
50%         499.500000    33.000000     NaN     2.000000     NaN             NaN             NaN    2319.500000    18.000000     NaN   NaN
75%         749.250000    42.000000     NaN     2.000000     NaN             NaN             NaN    3972.250000    24.000000     NaN   NaN
max         999.000000    75.000000     NaN     3.000000     NaN             NaN             NaN   18424.000000    72.000000     NaN   NaN

Missing Values:
Unnamed: 0          0
Age                 0
Sex                 0
Job                 0
Housing             0
Saving accounts     0
Checking account    0
Credit amount       0
Duration            0
Purpose             0
Risk                0
dtype: int64

Duplicate Rows:
0

Data Types:
Unnamed: 0          int64
Age                 int64
Sex                   str
Job                 int64
Housing               str
Saving accounts       str
Checking account      str
Credit amount       int64
Duration            int64
Purpose               str
Risk                  str
dtype: object
```

```
Target Variable: Risk

Target Values:
Risk
bad     700
good    300
Name: count, dtype: int64

Credit Risk Class Distribution:
Risk
bad     700
good    300
Name: count, dtype: int64

Credit Risk Percentage:
Risk
bad     70.0
good    30.0
Name: proportion, dtype: float64

Gender Representation:
Sex
male      690
female    310
Name: count, dtype: int64

Gender Percentage:
Sex
male      69.0
female    31.0
Name: proportion, dtype: float64

Credit Risk by Gender:
Risk    bad  good
Sex
female  201   109
male    499   191

Credit Risk Percentage by Gender:
Risk          bad       good
Sex
female  64.838710  35.161290
male    72.318841  27.681159

Age Group Distribution:
AgeGroup
18-25    123
26-35    310
36-50    260
51-65    201
66+      106
Name: count, dtype: int64

Credit Risk by Age Group:
Risk      bad  good
AgeGroup
18-25      86    37
26-35     217    93
36-50     182    78
51-65     141    60
66+        74    32

Credit Risk Percentage by Age Group:
Risk           bad       good
AgeGroup
18-25    69.918699  30.081301
26-35    70.000000  30.000000
36-50    70.000000  30.000000
51-65    70.149254  29.850746
66+      69.811321  30.188679

Housing Category Representation:
Housing
own     713
rent    179
free    108
Name: count, dtype: int64

Credit Risk by Housing Category:
Risk     bad  good
Housing
free      64    44
own      527   186
rent     109    70

Credit Risk Percentage by Housing Category:
Risk           bad       good
Housing
free     59.259259  40.740741
own      73.913043  26.086957
rent     60.893855  39.106145

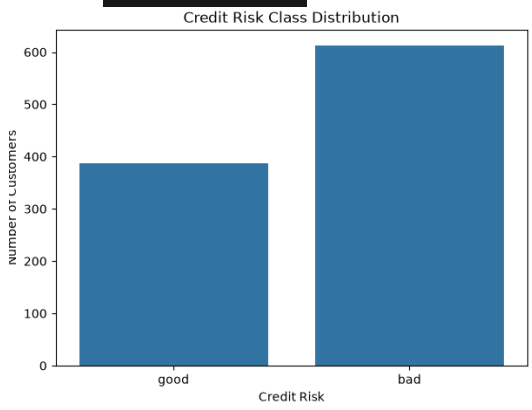
========== FINAL SUMMARY ==========

Overall Risk Distribution:
Risk
bad     70.0
good    30.0
Name: proportion, dtype: float64

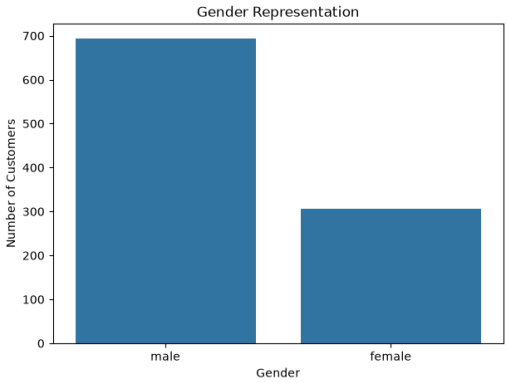
Risk by Gender:
Risk          bad       good
Sex
female  64.838710  35.161290
male    72.318841  27.681159

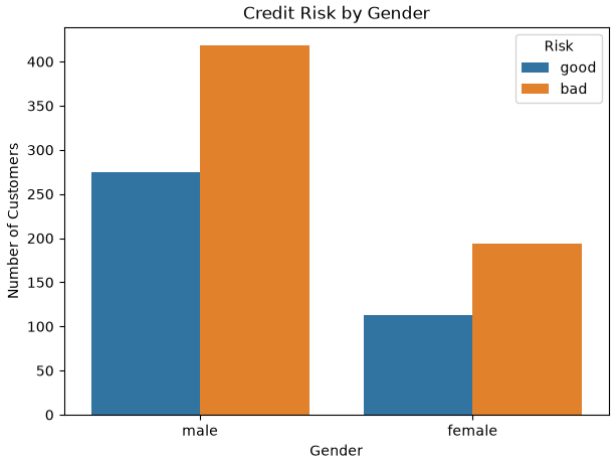
Risk by Age Group:
Risk           bad       good
AgeGroup
18-25    69.918699  30.081301
26-35    70.000000  30.000000
36-50    70.000000  30.000000
51-65    70.149254  29.850746
66+      69.811321  30.188679

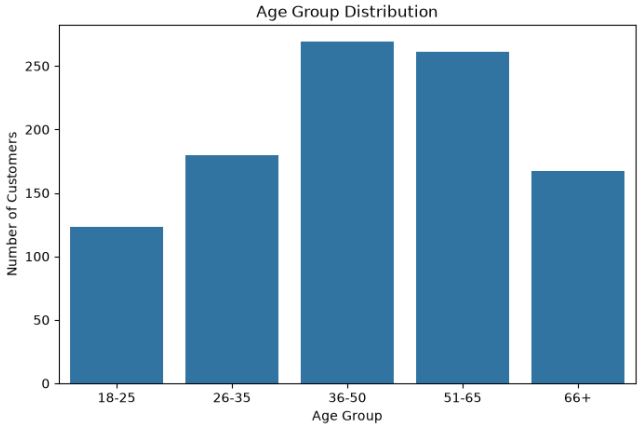
Risk by Housing:
Risk           bad       good
Housing
free     59.259259  40.740741
own      73.913043  26.086957
rent     60.893855  39.106145
```

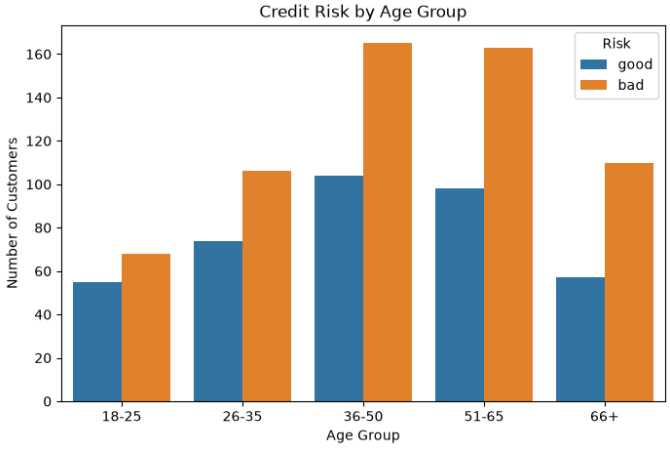


```
df["AgeGroup"] = pd.cut(df["Age"], bins=bins, labels=labels
, include_lowest=True)
print("\nAge Group Distribution:")
print(df["AgeGroup"].value_counts().sort_index())
plt.figure(figsize=(8,5))
sns.countplot(data=df, x="AgeGroup", order=labels)
plt.title("Age Group Distribution")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.show()
# 9. Analyze Credit Risk by Age Group
age_risk = pd.crosstab(df["AgeGroup"], df["Risk"])
print("\nCredit Risk by Age Group:")
print(age_risk)
plt.figure(figsize=(8,5))
sns.countplot(data=df, x="AgeGroup", hue="Risk", order=labels)
plt.title("Credit Risk by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.show()
age_risk_percentage = pd.crosstab(df["AgeGroup"], df["Risk"],
normalize="index") * 100
print("\nCredit Risk Percentage by Age Group:")
print(age_risk_percentage)
# 10. Analyze Housing Category Representation
print("\nHousing Category Representation:")
print(df["Housing"].value_counts())
plt.figure(figsize=(7,5))
sns.countplot(data=df, x="Housing")
plt.title("Housing Category Representation")
plt.xlabel("Housing Category")
plt.ylabel("Number of Customers")
plt.show()
# 11. Analyze Credit Risk by Housing Category
housing_risk = pd.crosstab(df["Housing"], df["Risk"])
print("\nCredit Risk by Housing Category:")
print(housing_risk)
plt.figure(figsize=(8,5))
sns.countplot(data=df, x="Housing", hue="Risk")
plt.title("Credit Risk by Housing Category")
plt.xlabel("Housing Category")
plt.ylabel("Number of Customers")
plt.show()
housing_risk_percentage = pd.crosstab(df["Housing"],
df["Risk"], normalize="index") * 100
print("\nCredit Risk Percentage by Housing Category:")
print(housing_risk_percentage)
# Additional Summary
print("\n========== FINAL SUMMARY ==========")
print("\nOverall Risk Distribution:")
print(df["Risk"].value_counts(normalize=True) * 100)
print("\nRisk by Gender:")
print(gender_risk_percentage)
print("\nRisk by Age Group:")
print(age_risk_percentage)
print("\nRisk by Housing:")
```

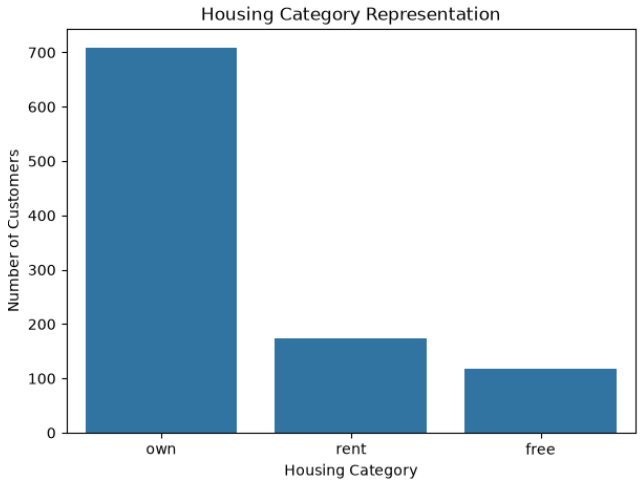


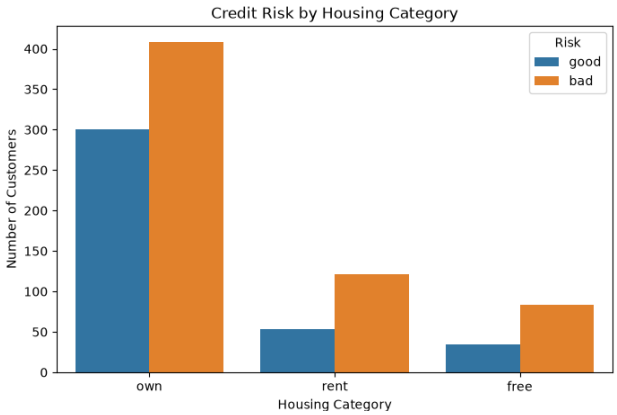



```
print(housing_risk_percentage)
```




## 12. Identify Possible Bias

The German Credit Risk dataset may contain several possible sources of bias:

- **Gender bias:** Credit-risk distribution may differ between male and female applicants.
- **Age bias:** Younger and older applicants may have different risk patterns.
- **Housing bias:** Home ownership, renting, or free housing can be associated with financial status.
- **Class imbalance:** Good-risk customers are generally more common than bad-risk customers.
- **Historical bias:** The dataset reflects historical lending decisions and may contain historical socioeconomic patterns.
- **Representation bias:** Some demographic groups may have fewer observations.

## 13. Impact on Machine Learning

Potential effects on an ML credit-risk model include:

1. The model may favor the majority risk class.
2. Minority-class applicants may receive poorer predictions.
3. Gender or housing information may act as a proxy for socioeconomic status.
4. The model may learn historical lending patterns instead of true creditworthiness.
5. Overall accuracy may look good while the model performs poorly on the **bad-risk** class.

For example, if most customers are classified as `good`, a model predicting `good` for nearly everyone can achieve high accuracy while being poor at identifying risky customers.

## 14. Suggested Solutions

Recommended approaches include:

- Check and handle missing values.
- Remove unnecessary identifier columns.
- Encode categorical variables correctly.
- Use **stratified train-test splitting**.
- Use class weights or resampling techniques when necessary.
- Evaluate with **precision, recall, F1-score, ROC-AUC**, and confusion matrix.
- Examine performance separately across gender, age, and housing groups.
- Compare false-positive and false-negative rates between groups.
- Avoid using sensitive/proxy variables when they are not justified.
- Use fairness-aware model evaluation before deployment.
- Keep the model focused on financially relevant variables rather than historical demographic patterns.

## 15. Final Conclusion

The German Credit Risk dataset is useful for studying **class imbalance and potential demographic bias in machine learning**. Credit risk can vary across gender, age, and housing categories. These differences may represent genuine relationships, historical socioeconomic patterns, or bias in the original lending data. Therefore, a credit-risk ML system should not be evaluated only by overall accuracy. **Class-wise performance and fairness across demographic groups should also be examined**.